0.1 What is a Simulator?

A simulator is a digital tool designed to reproduce the behavior of a physical system from either a mathematical model or a physical model.

Its purpose is to study, test, and visualize how a robot behaves without the need to manipulate the real system.

In mobile robotics, a simulator allows you to:

- Predict the trajectory and forces acting on the robot as a function of its control input,
- Validate a control or navigation algorithm,
- Or simply visualize the robot's motion in a 3D environment.

0.2 Mathematical Representation of a Dynamic System

Any dynamic system can be represented by a set of ordinary differential equations (ODEs):

$$\dot{x}(t) = f(x(t), u(t), w(t), t)$$
$$y(t) = h(x(t), u(t), w(t), t)$$

where:

- $x(t) \in \mathbb{R}^n$ is the state vector (e.g., position, velocity, orientation, etc.),
- $u(t) \in \mathbb{R}^m$ is the control vector,
- $w(t)$ represents external disturbances or unmodeled dynamics,
- $y(t) \in \mathbb{R}^n$ is the output vector (measured quantities, sensor outputs),
- f and h are non linear functions that represent, respectively, the system dynamics and the measurement model.

The function f governs the evolution of the state over time given the current state, input, and possible disturbances.

Numerical Simulation: Integrating the Dynamic Model

A numerical simulator cannot handle continuous time directly.

Instead, it approximates the continuous-time trajectory $x(t)$ with a discrete sequence of states x_k at discrete time instants:

$$t_k = k.dt$$

where dt is the time step.

This process is called discretization.

1 Example: Euler Integration

The Euler method is the simplest numerical integration scheme.

It assumes that over a short time step dt, the derivative $x\dot{x} = f(x, u)$ is approximately constant.

Thus, the new state is obtained by a linear extrapolation from the current derivative:

$$x_{k+1} = x_k + f(x_k, u_k)\, dt$$

In other words, the next state is estimated by moving along the instantaneous slope $f(x_k, u_k)$ for a short interval of time dt.



Figure 1: Example of dynamic 3D simulation

These exercises have been written to train your real-time dynamical simulation skills with Unity game Engine and ROS2 robotics middleware.

2 Exercices:

Warning: in Unity, the frame is defined as indirect. With a horizontal plane $(\overrightarrow{x}, \overrightarrow{z})$ and an upward vertical axis $\overrightarrow{y}$.

However, in robotics, we classically define a 3D frame as direct with a horizontal plane $(\overrightarrow{x}, \overrightarrow{y})$ and a vertical upward axis $\overrightarrow{z}$.

In the following exercise, we will use this classic definition so you will have to take it into account while tackling objects 3D position in your Unity scene.

Exercice 0: Unity+ROS2 installation and first use

Check this link to be able to install and use Unity, ROS2 Jazzy jointly and test it.

https://youtu.be/6t4ymVSfwPo

2.1 Exercice 1: Bouncing Ball

Consider a 3D space $S \in \mathbb{R}^3$. At the initial time t_0, a bouncing ball denoted B with a mass m is dropped from z_0.

- In your Unity scene, place a 2D plane at coordinates $X_p = \begin{bmatrix} 10 \\ 22 \\ 1 \end{bmatrix}$;
  - Your plan should be a square with a $5m$ sized side;
  - Add a rigidbody, and uncheck "use gravity" box for this object.
- Then, place a ball B at $z_0^b = 3m$ above the plane.
  - With a mass $m_b = 40g$ and a radius $r_b = 1m$;
  - Add a rigidbody
- Modify the pose of your camera to better see the scene in the game and start your simulation. You should see the ball falling and staying above the plane.
- Then, place a square Q at $z_q^0 = 4m$ above the plane. With a mass $m_q = 20g$ and size $c_q = 3m$;
- Finally, import a 3D .obj file O into your Unity scene. Then, place it at $z_o^0 = 3m$ above the plane. Add a mass $m_o = 20g$. Its size does not matter;
- Change bounciness, initial height, and materials of the different objects in the scene to observe differences;
- Finally, change the plane's pitch until your objects slide over the surface.

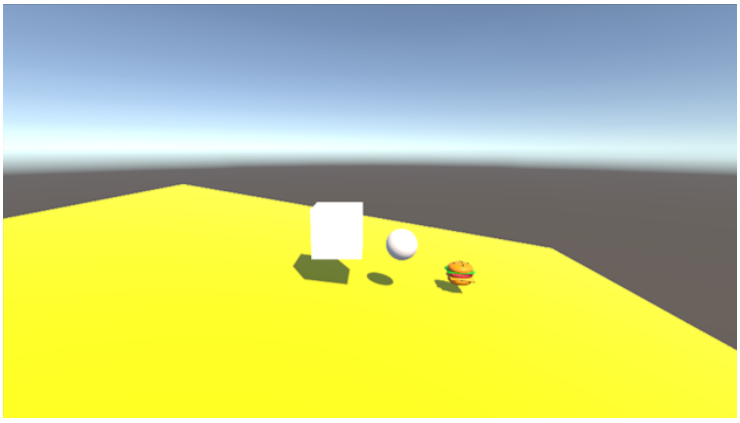


Figure 2: Summary illustration of the proposed approach

A step by step video of this exercise can be found here: https://youtu.be/uea0uhqWZo8

2.2 Exercice 2: Dubins' car

2.2.1 Mathematical tools

Consider an underactuated 2D vehicle V that faces maximum turning rate constraints such as a classic car. This vehicle cannot turn without translating. Leister Dubins developed in 1957 a kinematic model detailed in [1]. Let us recall the 2D kinematic Dubins' car model.

In a fixed 2-D frame $\mathcal{R}$. The following notations will be used throughout this paper:

- $X(t) = [x(t), y(t)]^T$: vehicle's position in $\mathcal{R}$;
- ψ: vehicle's heading;
- v: vehicle's speed;
- R: vehicle's minimum turning radius;

$$\begin{cases} \dot{X}(t) = \begin{pmatrix} u_1 \cos(\psi(t)) \\ u_1 \sin(\psi(t)) \end{pmatrix}, \\ \dot{\psi}(t) = u_2(t). \end{cases} \tag{1}$$

where $u_1(t) \in [0, v_{max}]$ is the forward speed control input and $u_2(t) \in \left[-\frac{R}{v_{max}}, \frac{R}{v_{max}}\right]$ is the turning speed control input.

2.2.2 Exercise

In this exercise, we will not use a Rigidbody but only a $C\#$ script modifying the car state at every step. The state of an object can easily be changed by modifying its transform.

- Place a plane and a car model in your Unity scene, add it a $C\#$ script;
- Add the Dubins' car model into your $C\#$ script. Do not forget that Unity object's frame is not standard and, particularly, is indirect;
- Use the Unity input system to be able to control $u_1$and u_2 at any time. Recently, Unity developed a new input system that is both more complex and complete. It is recommended to learn how to use it.

Remark : In this exercise, we implement a kinematic $C\#$ script to move an object by its transform. However, this way of doing it does not allow the user to collide with other 3D object of the scene without creating all the codes manually. The next exercise proposes using the Unity dynamics model.

A step by step video of this exercise can be found here: https://youtu.be/3w-u7QyUMXs

2.3 Exercise 3: USV Forces — A Naive Approach

Let us create a 3D simulation of a floating USV with two thrusters Note that in a real scenario, a boat stays stable thanks to its geometry which creates buoyancy compensating its roll and pitch. In this exercise, we simplify the model not to face this hydrodynamic issue.

- Place a 2D plane with a blue transparent material to represent the water at altitude $z_{water} = 0m$.
- Place your USV in the scene and give it a rigid-body with a mass $m = 5000kg$. Note that the weight has already been implemented by Unity thanks to your rigid-body. If you launch the simulation, the USV will sink vertically. In your rigid-body, check boxes to stop the roll and pitch.
- Place two objects without rendering as children of your USV where the thrusts should be, and take care with the orientation of the objects. The USV and thrusts should face the Z-direction, as Unity considers it as the "forward" direction.
- Add a script to your USV to add forces. The buoyancy force should be applied to the entire object at CoM.

$$rb.AddForce(F_{Thrust}, ForceMode.Force)$$

- In the previous script, add the thrust at the position of your thruster invisible object forces.

  $rb.AddForceAtPosition(F_{Thrust}, object.transform.position, ForceMode.Force)$;

- Modify the value of F_{Thrust}, the linear and angular drags for your simulations to look like real.

To go further: add a skybox.

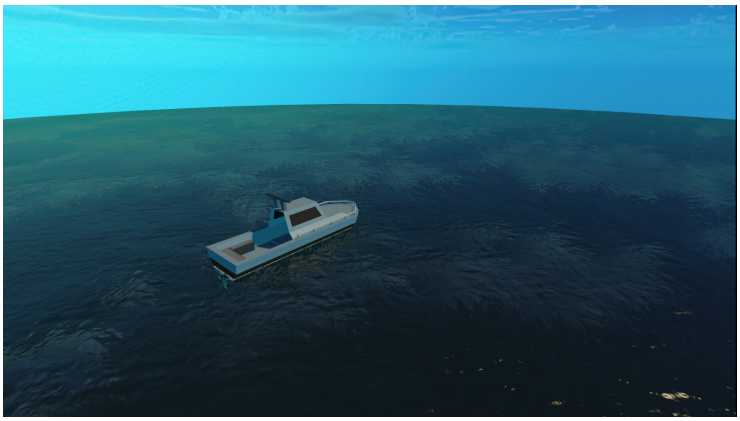


Figure 3: illustration of the forces applied to USV

2.4 Exercise 4: Floating Cube

Consider a 3D space $S \in \mathbb{R}^3$. A cube C with mass m is placed above a water surface. The objective is to simulate its motion in Unity by applying gravity and an approximate Archimedes' buoyancy force.

- In your Unity scene, create a water surface at $z_w = 0$.
- Place a 3D cube C above the water surface and add a `Rigidbody` component. Enable gravity.
- Subdivide the cube's volume into a set of N points distributed along the x, y, and z axes.
- At each physics simulation step, identify the points located below the water surface. These points are considered immersed. Let N_{imm} denote the number of immersed points.
- Compute the buoyancy force using the proportion of immersed points:

$$\vec{\Pi}_A = \rho V g \frac{N_{imm}}{N} \vec{u} \tag{2}$$

where:

- ρ is the water density, expressed in kilograms per cubic meter kg/m^3.
- V is the total volume of the object, expressed in cubic meters (m^3).
- g is the gravitational acceleration, approximately $9.81, m/s^2$ on Earth.
- N is the total number of sampled points.

- N_{imm} is the number of immersed points.
- $\vec{u}$ is the unit vector pointing vertically upwards.

The last two steps involving volume subdivision and immersion detection may be challenging. Take your time to understand the underlying concepts and implement them carefully.

Compute the application point of the buoyancy force as the barycenter of the immersed points:

$$\vec{p}_A = \frac{1}{N_{imm}} \sum_{i \in I_{imm}} \vec{p}_i \tag{3}$$

where I_{imm} is the set of indices of the immersed points and $\vec{p}_i$ is the position of point i in world coordinates. If no points are immersed, do not apply any buoyancy force.

- Apply the resulting force to the object's `Rigidbody` at the computed application point using `Rigidbody.AddForceAtPosition()`.
- Run the simulation and observe how the object falls into the water and floats.
- Change the object's mass, the water density, and the number of sampled points to observe their effects on the simulation.
- Test different object shapes and initial orientations. You may use a cube, a cylinder, or a sphere. The subdivision strategy may vary depending on the shape. For more complex objects, consider decomposing them into simpler geometric primitives, such as cubes, spheres, or cylinders.

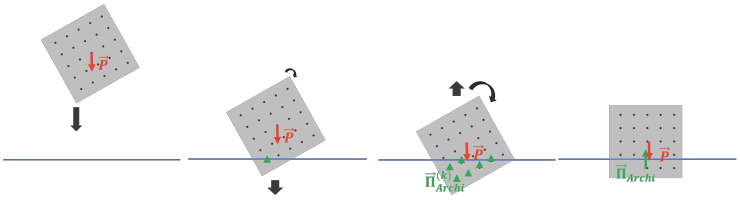


Figure 4: Summary illustration of the proposed approach

2.5 Exercice 5: Lidar

Create an object called Lidar. Its appearance does not matter, a simple cube is sufficient.

Attach a C# script to it that casts rays in all directions, retrieves the distance to the first object hit by each ray, and stores the results in a list.

To better visualize what the LiDAR detects, place a particle (lighter on the game engine) or an object (easier to implement, but much more resource-intensive and less visually appealing) at each ray-object intersection.

Finally, adapt your code to publish the points detected by the LiDAR on a ROS 2 topic named /pointcloud, using a PointCloud message.

Note that PointCloud2 is also available, but is more complex to implement. Alternatively, you can define your own custom message to better suit your specific requirements.

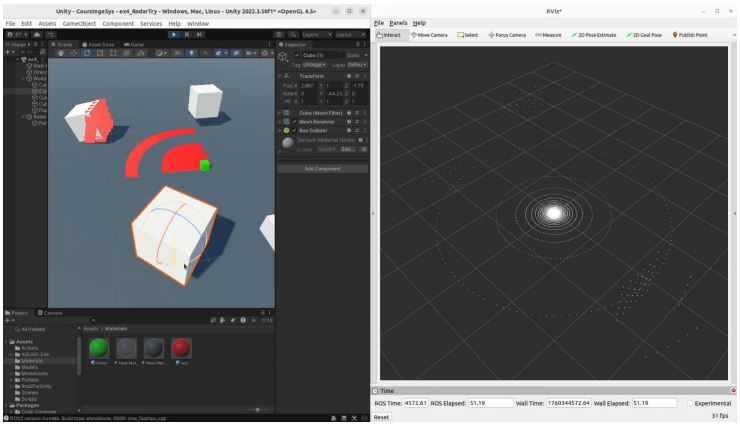


Figure 5: illustration of the radar scene

2.6 Exercice 6: Camera publisher

Copy your favorite scene and add it a camera in your new Unity scene. Its position and orientation should be chosen to observe the environment from the robot's point of view.

Attach a C# script to the camera that captures the image rendered by the Unity camera at regular intervals. This exercise may be more challenging than the previous ones. As you progress through the exercise book, fewer hints will be provided to guide you towards the solution. You are free to use any online resources to help you complete the exercise.

Then, use the ROS-TCP-Connector to publish the captured images to a ROS 2 topic named /camera/image.

- Retrieve the image rendered by the Unity camera;
- Convert the image data into a format compatible with ROS 2 messages;
- Create a publisher using the appropriate ROS-TCP-Connector message type;
- Publish the camera images at a fixed rate on /camera/image;
- Finally, use a ROS 2 subscriber to retrieve and display the images published by Unity.

You can use tools such as rqt_image_view to visualize the camera stream. Make sure that the image encoding, resolution, and message format are consistent between Unity and ROS 2.

Experiment with different camera positions, orientations, and resolutions to observe their effects on the images received by ROS 2

2.7 Exercise 7: Inertial Navigation System

Create an Inertial Navigation System (INS) sensor in your Unity scene to estimate the robot's motion and orientation.

Attach a C# script to the robot that retrieves its motion data from the Unity physics engine and computes the measurements provided by an inertial sensor.

Then, use the ROS-TCP-Connector to publish the measurements to ROS 2 topics.

- Retrieve the robot's orientation from its Unity Transform and convert it into a suitable representation for ROS 2;
- Retrieve the robot's linear and angular velocities from its Rigidbody;
- Estimate the linear acceleration using the variation of velocity over time, taking gravity into account if necessary;
- Create publishers using the appropriate ROS-TCP-Connector message types;
- Publish the orientation, angular velocity, and linear acceleration using an 'Imu' message on '/imu/data';
- Finally, use a ROS 2 subscriber to retrieve and display the measurements published by Unity.

Note that a real INS relies on inertial sensors such as accelerometers and gyroscopes to estimate the vehicle's motion. In this exercise, you will use the ground-truth data available in Unity to simulate these measurements. You may introduce measurement noise and sensor bias to make the simulated sensor more realistic. For example, add Gaussian noise to the angular velocity and linear acceleration, or introduce a constant bias to simulate sensor imperfections.

Make sure to account for the differences between Unity and ROS 2 coordinate conventions when converting orientations and velocities.

Experiment with different robot trajectories, rotations, noise levels, and sensor biases to observe how the inertial measurements evolve over time.

2.8 Exercise 8: GNSS

Create a GNSS sensor in your Unity scene to estimate the position of a robot in the environment.

Attach a C# script to the robot that retrieves its position and publishes measurements to a ROS 2 topic named '/gnss/fix', using the ROS-TCP-Connector.

- Retrieve the robot's position from its Unity Transform;
- Convert the position into a suitable representation for ROS 2 and publish it using a 'NavSatFix' message;
- Use a ROS 2 subscriber to retrieve and display the GNSS measurements.

Note that GNSS positions are generally expressed as latitude, longitude, and altitude, whereas Unity uses Cartesian coordinates. You will need to define a reference position and implement a suitable coordinate transformation, or publish local Cartesian coordinates using a 'Point' or 'Pose' message.

Finally, add noise to the position measurements to simulate GNSS inaccuracies. Experiment with different noise levels and robot trajectories to observe their effects on the measurements.

References

[1] L. E. Dubins, "On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents," *American Journal of Mathematics*, vol. 79, no. 3, pp. 497–516, 1957. [Online]. Available: https://www.jstor.org/stable/2372560